

Stack & Flow (ehem. ᚠᚱᚱ = NYT, nützliche Datenflüsse)

Nyt (die nützliche) Flussname im Lied der Grímnismál (Edda):

https://de.wikipedia.org/wiki/Liste_der_Fl%C3%BCsse_im_Lied_Gr%C3%ADmnism%C3%A1l

Stack & Flow soll eine minimalistische formale Sprache zur semantischen Auszeichnung von Texten, zur funktionalen Formulierung von Algorithmen und zur generatorischen Beschreibung von Diagrammen, Bildern und Fräskopfbahnen werden.

Grundlagen

Runen als Präfix

Alle für den Parser unterscheidbaren Strukturen erhalten ein Präfix in Form einer nordischen **Rune**.

Die *Runen* werden in keiner heute mehr existierenden Sprache genutzt. Damit sind die Präfixe, durch die Sprachstrukturen kenntlich werden, eindeutig von Textdaten unterscheidbar.

Kommentare +

+ schließt den Rest vom Parsen aus. Damit können nach + beliebige Kommentare notiert werden.

Literale elementarer Datentypen

Präfixe für die Notation von Zahlenwerten

Eine Gleitpunktzahl wie **3.14** ist eine kulturspezifische Notation (**en-US**).

Um die Notation von Zahlenwert von einer textuellen und kulturspezifischen Präsentation in einer Sprache zu unterscheiden, werden diese in **Stack & Flow** stets durch ein spezielles *Präfix* explizit gekennzeichnet.

 Zahlen können wie z.B. **R _Zähler_ _Nenner_** eine listenartige Struktur darstellen, sind aber keine Listen. Die einzelnen Partikel wie im Beispiel **_Zähler_** und **_Nenner_** dürfen nur Konstanten sein, wie **R 1 2**, jedoch keine Ausdrücke!

Numerische Datentypen

Die Notationsformen für Zahlenwerte haben Beschränkungen bezüglich der Genauigkeit. Deshalb korrespondieren die Notationsformen auch mit Teilmengen von $\mathbb{Q}$. Diese Teilmengen werden *Numerische Datentypen* genannt.

Die numerischen Datentypen werden durch Kombination des speziellen Präfixes für eine Notation (z.B. **K**) mit dem allgemeinen Datentyp-Schalter **T** verbunden zum Datentyp Symbol **KT**.

T schaltet allgemein die Evaluierung einer Liste in die Evaluierung einer Typdeklaration um.

T alleine steht für jeden beliebigen Datentyp.

Basis des Zahlensystems $\mathbb{B}$

Zahlen können über verschiedenen *Basen* dargestellt werden. So kann die **26** dargestellt werden dekadisch mit der Basis **10** als **26**, hexadezimal mit der Basis **16** als **1A**, und binär mit der Basis **2** als **LL0L0**.

Die Basis kann in einen numerischen Typ explizit definiert werden mit dem Präfix **B**

```
+ Basis 2
B2

+ Basis 10- kann in der Regel entfallen, da Default
B10

+ Basis 16
B16
```

Kardinalzahlen K

K ist das Präfix für ganze Zahlen:

```
K 1          ⇔ 1
K -123       ⇔ -123
K B16 AFD    ⇔ nat. Zahl zur Basis 16 (hex)
K B2 L00LLL  ⇔ nat. Zahl zur Basis 2 (dual)
K M          ⇔ + Unendlich
K -M         ⇔ - Unendlich
```

K^T ist der Datentyp für Kardinalzahlen.

Gebrochen Rationale Zahlen R

R ist das Präfix für gebrochen rationale Zahlen. Diese bestehen aus einem *Nenner* und einem *Zähler*, getrennt durch ein Leerzeichen:

1. **R** *_Zähler_* hier ist der Nenner stets 1
2. **R** *_Zähler_* *_Nenner_*
3. **R** *_Ganzzahlig_* *_Zähler_* *_Nenner_*

Beispiele:

```
R 2          ⇔ 2/1 = 2.0
R 1 2        ⇔ 1/2 = 0.5
R 1 2 3      ⇔ 1 2/3 = 1.666
R -4 16      ⇔ -4/16 = -0.25
R B2 -L00 L0000 ⇔ -4/16 = -0.25 im binärsystem
```

Die rationalen Zahlen können z.B. als Zoll- Maße genutzt werden

R^T ist der Datentyp für gebrochen rationale Zahlen.

Gleitpunktzahlen $\mathbb{F}$

$\mathbb{F}$ ist das Präfix für rationale Zahlen in der Gleitpunkt- Darstellung. Vor- und Nachkomma- Stellen bilden die beiden Elemente einer Liste. Kulturspezifische Separatoren wie `,` oder `.` sind damit überwunden.

```
 $\mathbb{F}$  3       $\iff$  3.0
 $\mathbb{F}$  3 14    $\iff$  3.14
 $\mathbb{F}$  -2 72   $\iff$  -2.72
 $\mathbb{F}$  -2 72 3  $\iff$  -2.72e3 = -2720
 $\mathbb{F}$  B2 -L00 L0000  $\iff$  -4,5 (binär)
```

$\mathbb{F}\mathbb{T}$ ist der Datentyp für Gleitpunkt- Zahlen.

Die Datentypen $\mathbb{R}\mathbb{T}$ und $\mathbb{F}\mathbb{T}$ sind kompatibel bzw. austauschbar: Ein $\mathbb{R}\mathbb{T}$ kann an ein $\mathbb{F}\mathbb{T}$ zugewiesen werden und umgekehrt.

Boolsche Werte $\mathbb{B}$

$\mathbb{B}$ ist das Präfix für boolsche Werte. Die beiden möglichen boolschen Werte werden durch die Namen **true** und **false** ausgedrückt:

```
 $\mathbb{B}$  true   $\iff$  True
 $\mathbb{B}$  false  $\iff$  False
```

$\mathbb{B}\mathbb{T}$ ist der Datentyp für boolsche Werte.

Strings $\mathbb{J}$

Strings sind Listen aus beliebigen Zeichen. Sie können auch Leerzeichen enthalten.

Strings, die keine Leerzeichen enthalten, können direkt notiert werden.

```
+ geschlossener String, enthält keine Leerzeichen
Hallo

+ geschlossener Strings, die einzelne Hierarchieebenen benennen
 $\mathbb{J}$  All Galaxieen Andromeda  $\mathbb{J}$ 
```

Enthalten *Strings* Leerzeichen, dann müssen sie in ein **S-Array**: $\mathbb{J}$... $\mathbb{J}$ gesetzt werden. $\mathbb{J}$ ist das Präfix für String- Listen.

Die Leerzeichen sind innerhalb eines String- Array geschützt.

Sehr Lange Strings können mittels $\mathbb{J}$ auf mehrere Zeilen umgebrochen werden.

```
+ String aus mehreren Wörtern. Die Leerzeichen sind geschützt
JHallo   WeltJ

+ Komplexe Texte als String, umgebrochen auf mehrere Zeilen mittels J
J Mit Strings können auch **Markdown** formatierte Texte geschrieben werden.J
So wird *Text* und *Logik* vollständig vermischt.J
```

JT ist der Datentyp für Strings.

Stringinterpolation

Werden in einem String Namensreferenzen eingesetzt, die beim Abruf des Strings evaluiert werden, dann liegt eine Stringinterpolation vor.

Sei **xattrib schöne** eine Namensbindung. Dann kann eine Stringinterpolation wie folgt definiert werden:

```
J Hallo xattrib Welt J
```

Diese wird dann evaluiert zu:

```
J Hallo schöne Welt J
```

Hierarchieen P

P (runic Fehu) ist das Präfix eines Pfades in einer Hierarchie. Der Pfad muß durch ein Listenendsymbol J (runic Q) abgeschlossen werden.

P K23 K10 K15 J ⇔ Kann z.B. eine Versionsnummer mit den drei Hierarchieebnen *Hauptversion*, *Nebenversion*, *Buildnummer* darstellen. Oder die Uhrzeit **23:10:15**. Oder das Datum **15.10.2023**.

P millingMachine circelMilling millDiscJ ⇔ Pfad in einem Namensraum

PT ist der Datentyp für Hierarchieen.

Informationen darstellen durch binden von Werten an Namen mittels x Operator

Informationen bestimmen den Ausgang von Entscheidungen. Entscheidungen manifestieren sich durch die Zuordnung/Belegung/Bindung von Werten an Systemparametern.

Die Bindung eines Wertes an einen Systemparameter wird **Attribut** genannt.

Mittels dem **Bind** Operator x (runic Othalan) kann in **Stack x Flow** ein Wert an einen Namen gebunden, und somit ein Attribut gebildet werden. Über den Namen ist der Wert dann referenzier- und abgerufbar.

Der Namen muss innerhalb seines *Namensraumes* (siehe unten) eindeutig sein. Ein typischer Namensraum ist die **Stack x Flow** Datei.

Attribute bzw Namensbindungen sind wie folgt aufgebaut: x <Name als String> <Wert>

Beispiele:

+ Konstante PI definieren

```
⊗PI ₣ 3 14
```

+ Liste der ersten fünf Primzahlen an einen Namen binden

```
⊗ersteFünfPrimzahlen ₢K2 K3 K5 K7 K11 ¶
```

Die Bindung eines Namens an einen Wert kann auch als **Attribut Wertepaar** betrachtet werden!

Naming ID's ₢

₢ ist das Präfix für eine *NamingID*. Eine *NamingID* ist ein global eindeutiger Name in Form einer **64bit GUID**.

An eine global eindeutige *NamingID* kann wie an einen lokalen *Namen* ein Wert gebunden werden in der Form `⊗₢ K B16 <Hex- Wert Naming ID> _Attribut-Wert_`.

Beispiele:

+ An die global gültige Naming ID 0x7ABC123 wird der Wert 3,1427 gebunden.

```
⊗₢ K B16 7ABC123 ₣ 3 1427
```

₢₣ ist der Datentyp für Namensreferenzen.

Namen in einen Wert auflösen mittels * (Ior)

Wurde an einen Namen ein Wert gebunden, dann kann überall, wo normalerweise der Wert eingesetzt wird, der Name eingesetzt werden. Dazu ist dem Namen das runic Ior * voranzusetzen:

+ Konstante PI definieren

```
⊗PI ₣3 14
```

+ Den Wert von ****PI**** an den synonymen Namen ****pie**** binden

```
⊗pie *PI
```

+ Den Wert der globalen mit Naming ID definierten Konstante ****PI**** an den synonymen Namen ****piGlob**** binden

```
⊗piGlob *₢ K B16 7ABC123I
```

Namensraum- Strukturen ⊗ ... ₢ ... ¶

Eine Menge von *Bind* Operationen können in Listen `₢ ... ¶` zusammengefasst werden. Innerhalb einer solchen Liste darf ein bestimmter Name stets nur einmal an einen Wert gebunden werden. Diese Listen werden **Namensraumstruktur**, oder kurz **Struktur** genannt.

+ Beschreibung einer Punktkoordinate durch eine Namensraumstruktur

```
₢ ⊗x ₣2 72 ⊗y ₣3 14 ¶
```

Die Struktur kann selber als Wert mittels Bind an einen Namen gebunden werden. So entsteht ein **Namensraum**

```
+ Namensraum mathematischer Konstanten
⌵MathConst
└
  ⌵PI  F3 14
  ⌵e   F2 72
└

+ Namensraum, der einen Punkt darstellt
⌵Punkt1
└
  ⌵x  F2 72
  ⌵y  F3 14
└
```

Hierarchische Namensraum Referenzen $\mathcal{N}\mathcal{P} \dots \mathcal{N}$

Die Namen in einem Namensraum sind außerhalb dieses nicht mehr eindeutig. Eine eindeutige Addressierung der Attribute aus einer Position im Code außerhalb eines Namensraumes werden *Hierarchische Namensraum Referenzen* benötigt. Diese haben folgenden Aufbau: $\mathcal{N}\mathcal{P} \langle \text{Name 1. Level} \rangle \langle \text{Name 2. Level} \rangle \dots \langle \text{Name N. Level} \rangle \mathcal{N}$.

Für den Zugriff auf den Werte ist er hierarchischen Referenz der *for* Operator $\ast$ voranzustellen: $\ast\mathcal{N}\mathcal{P} \langle \text{Name 1. Level} \rangle \langle \text{Name 2. Level} \rangle \dots \langle \text{Name N. Level} \rangle \mathcal{N}$.

Beispiel

```
+ Organisation einer mathematischen Bibliothek
⌵Math
└
  ⌵Const
  └
    ⌵PI  F3 14
    ⌵e   F2 72
  └
└

+ Zugriff auf PI
 $\ast\mathcal{N}\mathcal{P}\text{Math Const PI}$ 
```

Namensräume auf Basis globaler Naming- IDs

Um abstrakte Naming- IDs besser zu handhaben, können sie an lesbare Namen mittels $\mathcal{N}$ gebunden, und diese lesbaren Namen in Namensraumstrukturen organisiert werden:

```

+ Organisation einer mathematischen Bibliothek, 2
⌵Math
P
  ⌵BasicFunctions
  P
    + Naming- IDs der math. Grundrechenarten werden an lokale Namen gebunden
    ⌵add *H K B16 ADDADD
    ⌵sub *H K B16 DE2323
  ↵
↵

+ Zugriff auf add
*H⌵Math BasicFunctions add↵

+ Hier wird über den hierarchischen Namen die Funktion aufgerufen
⌵H⌵Math BasicFunctions add↵ K1 K2
+ ⌵sum ist nur innerhalb des Siegel - Zweiges sichtbar
↳ ⌵sum ⌵print ↳ *sum ist die Summe aus 1 und 2 ↵

```

Arrays ⌵

Arrays sind Listen von Werten. Die Werte können primitiv oder komplex sein.

⌵ (runic Y) ist das Präfix, welches die Liste eines Arrays eröffnet. ↵ beendet die Liste.

```

+ Array mit den ersten fünf Primzahlen
⌵ K2 K3 K5 K7 K11 ↵

+ Array mit zwei Koordinaten
⌵
  P ⌵x ⌵2 72 ⌵y ⌵3 14 ↵
  P ⌵x ⌵5 3 ⌵y ⌵1 7↵ ↵
↵

+ Array aus Daten verschiedener Typen
⌵
  P ⌵x K2 ⌵y K3 ↵
  K13
  ↳ Summe aus a² und b² ↵
↵

```

⌵T ⌵red ⌵green ⌵blue ↵ steht für einen Aufzählungstyp/Set: Eingesetzt werden dürfen nur die im Array aufgelistete Werte.

Zugriff auf Array Elemente

Auf einzelne Elemente eines Arrays kann mittels Operator ⌵↑ _array_ _index_ ↳ _Ergebnis_ zugegriffen werden.

Dieser hat als Parameter den **0** basierte Index und das *Array*, aus dem der Wert zu entnehmen ist.

Soll im Falle eines Zugriffs auf ein nicht vorhandenes Element durch einen zu kleinen, oder zu großen Index keine Ausnahme, sondern eine benutzerdefinierte Fehlerbehandlung starten, dann ist `⌈↑ _array_ _index_ ⌋ _errIndexOutOfRangeHandler_ ↵ _Ergebnis_` einzusetzen.

Beispiele (hier enthalten Array selber wieder Arrays)

```
+ Array mit Elemente, die selber Arrays sind
&a1
⌈
  ⌈ K1 K2 ↵
  ⌈ K3 K4 ↵
↵

+ Hier gilt: *e1 == ⌈ K1 K2 ↵
⌈↑ &a1 1 ↵ *e1

+ Hier gilt: *e2 == ⌈ K3 K4 ↵
⌈↑ &a1 2 ↵ *e2
```

Einbetten von Array in Array mittels Expand X Operator

Mittels des Expand- Operator X kann der Inhalt eines Array in ein anderes eingebettet werden

```
+ Array mit Elemente, die selber Arrays sind
&a2
⌈
  K1
  X⌈ K2 K3 ↵
  K4
↵

+ Hier gilt: *e1 == K1
⌈↑ &a2 1 ↵ *e1

+ Hier gilt: *e2 == K2
⌈↑ &a1 2 ↵ *e2
```

Häufig benutzte Array- Operationen

Im folgenden werden Operationen auf Array beschreiben, die häufig in **LLP** einzusetzen sind.

Pop

`⌈pop ⌈ a b ... ↵` entnimmt das erste Element von Links aus dem Array.

```

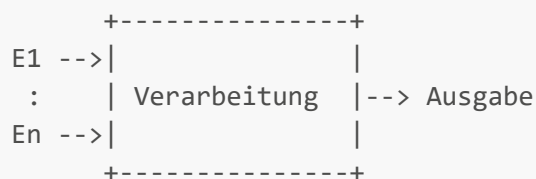
&ar1 ʀ a b ʀ

M↑pop &*ar1
↳ &res          + &*res == a
M M↑log ʀ      + loggt ʀ b ʀ

M↑pus &*ar1 ʀ
M M↑log ʀ      + loggt ʀ b ʀ

```

EVA: Eingabe, Datenverarbeitung, Ausgabe



Dieses uralte Prinzip der elektronischen Datenverarbeitung wird wieder in den Fokus gestellt. Die Verarbeitung von Daten erfolgt durch einzelne, benannte *Verarbeitungsstufen* (kurz *Stufe*). Jede Verarbeitungsstufe wird mit der Rune **CALC** λ eingeleitet, ihr folgt der Name der Verarbeitungsstufe, die Eingangsparameter, die Verarbeitungszweige und schließlich der Ausgang, der mit der Rune **EOLHX** Υ gekennzeichnet wird.

```

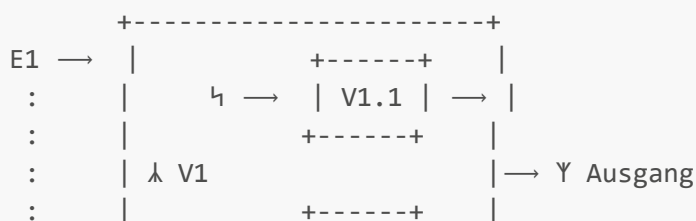
+ Syntaktischer Aufbau einer Verarbeitungsstufe
λ _NameVStufe_ _E1_ ... _En_
↳ _Verarbeitungsfunktion_im_SIEGEL_Zweig_
ξ _Verarbeitungsfunktion_im_SOWILO_Zweig_
Υ _Abschluss_oder_Folge_Funktion_am_Ausgang_

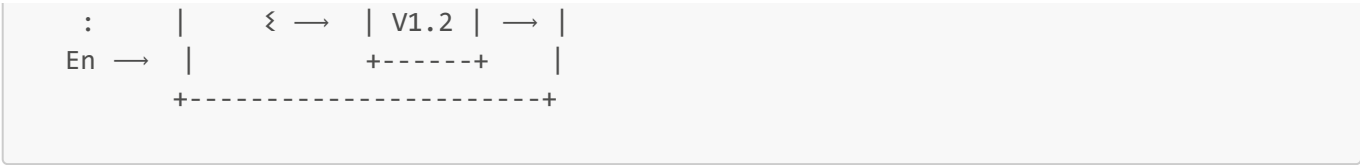
```

In **NYT** kann die Verarbeitung in einer Stufe stets in zwei alternative Pfade erfolgen. Damit wird das grundlegende Prinzip der Verzweigung eingeführt.

- ↳ SIEGEL Zweig
- ξ: SOWILO Zweig

Am Ende müssen aber beide Pfade wieder am Ausgang zu einem Pfad zusammengeführt werden. Damit können komplexe, jedoch strukturierte Datenflussgraphen konstruiert werden:





Eingangswerte/Paramter

Jede Stufe kann parametrierbar sein. Die Parameter (oder Eingangswerte) werden auf dem Stapelspeicher bereitgestellt. Der Stapelspeicher kann unmittelbar nach dem Stufenamen mit den benötigten Parametern befüllt werden durch eine Parameterliste: `λstufenNamen _p1_ ... _pn_`. Die Parameter werden dabei von rechts nach links auf dem Stapelspeicher des Laufzeitsystems abgelegt.

Eine einfache Verarbeitungsstufe, die dieses Prinzip direkt auszunutzt, ist die **push** Stufe. Sie legt alle Eingangsparameter unverändert auf dem Stapel des Laufzeitsystems ab:



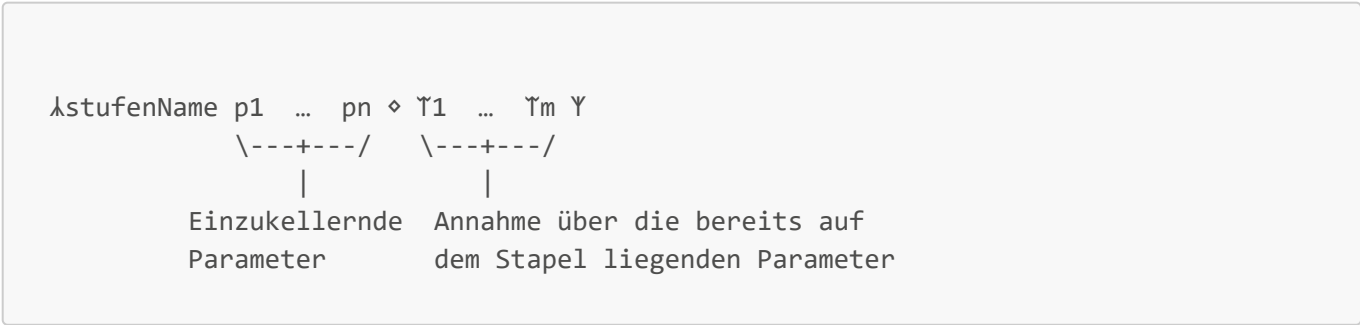
Annahmen zum Stapelspeicher definieren

Da jede Stufe ihre Parameter vom Stapel liest, muss sichergestellt werden, dass auch alle benötigten Parameter auf dem Stapel für die Stufe bereitstehen. Die Prüfung des Stapelspeichers erfolgt durch die Stufe zur Laufzeit. NYT stellt zudem eine generische Implementierung für solche Prüfungen bereit durch

Musterbelegungen:

Def Musterbelegung: ist eine Liste von Typnamen nach der INGWAZ Rune: `◊ T1 ... Tm Y`. Der erste Typname `T1` bezeichnet dabei den Datentyp des ersten Wertes auf dem Stapelspeicher, der zweite `T2` den des zweiten Wertes auf dem Stapelspeicher usw..

Die **Musterbelegung** kann an die Parameterliste einer Stufe angehängen werden, und definiert eine Annahme über die Belegung des Stapelspeichers vor dem Einkellern der Parameter einer Stufe:



Wenn eine **Musterbelegung** nicht zutrifft, dann wird eine Fehlermeldung erzeugt und auf dem Stapel abgelegt. Anschließend wird im **ξ (Sowilo)** Zweig der Stufe fortgesetzt.

 **Achtung:** Die Musterbelegung scheint einer formalen Parameterliste einer Prozedur in einer Programmiersprache wie **C#** zu entsprechen. Jedoch handelt es sich hier um ein automatisiertes Prüfverfahren für die Stapelspeicherbelegung zur Laufzeit (keine Prüfung zur Entwurfszeit via Compiler!), die beim konkreten Start der Stufe stattfindet. Es kann deshalb für verschiedene Stufenstarts auch verschiedene Musterbelegungen geben:

```
λpush K77Υ
```

+ Hier wird eine Musterbelegung von einer Kardinalzahl auf dem Stapelspeicher angenommen.

```
λadd K88 ◊ KΥ
```

```
ξ λprintΥ
```

```
Υ
```

+ Hier wird eine Musterbelegung von zwei Kardinalzahlen auf dem Stapelspeicher angenommen.

```
λadd ◊ KΥ KΥ
```

```
ξ λprintΥ
```

```
Υ
```

Zweige

Eine Methode verarbeitet die übergebenen Parameter. Danach gibt es drei Möglichkeiten der Programmfortsetzung:

1. Es wird im **λ (Siegel) Zweig** fortgesetzt
2. Es wird im **ξ (Sowilo) Zweig** fortgesetzt
3. Es wird sofort zum Stufenausgang **Υ (Eolhx)** gesprungen und diese damit formal beendet

In den Fällen 1 und 2 wird nach Durchlauf der Zweige ebenfalls am Ausgang **Υ** abgeschlossen.

Beispiele:

+ Wurzel aus einer Zahl a ziehen

```
λ SQR a 1
```

λ **_op_auf_va_** + Hier wird die $\sqrt{}$ von a bereitgestellt

ξ **_Fehlerbehandlung_** + z.B. im Fall $a < 0$

Υ **_Abschlussfunktion_** + Hier wird der Stapelspeicher Nach Ausführung von λ oder ξ bereitgestellt

Bereitstellung der Ergebnisse in den Zweigen

```
λ m a b c      --+---+---+
               a | b | c | -----+-----+      + Ablage der Parameter auf dem
Stapelspeicher
```

```
               --+---+---+      |      |
               -----+      |      |
```

```

+--- h s      m(a, b, c) | <-+      |      + Ergebnis der Methode s im h Zweig
bereitstellen
|
|      -----+      |
|      -----+      |
| +-- ξ e      m(a, b, c) | <-----+      + Ergebnis der Methode e im ξ Zweig
bereitstellen
| |
| |      -----+
| |
| |      -----+
+--> γ      s(m(a, b, c)) oder e(m(a, b, c)) | + Ergebnis vom h oder ξ Zweig
bereitstellen
      -----+

```

Beispiel: Berechnen der Quadratwurzel

```

λ input J a² = 1
h xaa
+ Ende von Input
γ

λ print J Es wird nun die Wurzel aus *aa gezogen 1

+ Start Wurzel ziehen (Inhalt von xaa wird auf den Stapel gelegt)
λ sqrt *aa

+ Weiterleiten des Ergebnisses an die Print- Methode. Achtung: Im AusgabeString
findet
+ String- Interpolation statt.
h print K2 J √ x*aa= 1 ♦ Kγ

+ Weiterleiten im Fehlerfall an die Print- Methode. Achtung: Im AusgabeString
findet
+ String- Interpolation statt.
ξ print K2 J √ x*aa ist konnte nicht ermittelt werden. Ursache: 1 ♦ Jγ

+ Hier werden die Ausführungspfade wieder zusammengeführt
γ print K1 J Programm √ beendet 1

```

Hintereinanderschalten von Stufen in Sequenzen

Verarbeitungsstufen können direkt hintereinander ausgeführt werden: `λV1γλV2γ...λVnγ`. Die Ausgaben der ersten landen dabei auf dem Stack, von dem sie die zweite Verarbeitungsstufe einlesen und weiterverarbeiten kann usw.

```

+ Input liest einen Wert von der Tastatur ein und legt ihn auf den Stack
λinput J gib eine ganze Zahl z ein. Der absolute Betrag |z| wird ermittelt ! 1 γ

+ Vergleichsoperator 0 > eingabe
λa_gt_b K0 ♦ Kγγ

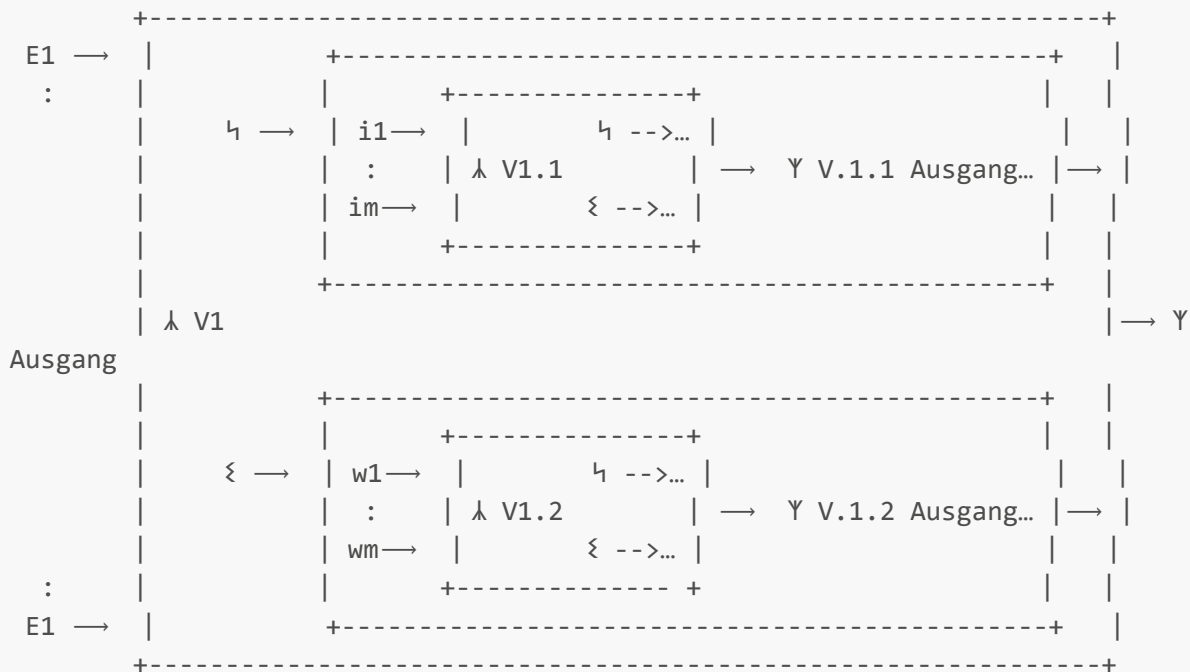
```

```
+ Auf dem Stack liegt das Ergebnis von 0 > eingabe
λifElse ◊ B T 4 λmul K-1 ◊ K T Y + Wenn eingabe < 0 ist, dann mit -1 multiplizieren

+ Print liest die nächsten beiden Werte vom Stack, und gibt sie aus.
λprint K2 J Der absolute Betrag |z| = 1 ◊ K T Y
```

Verschachtelung von Stufen

In den 4, 5 und 6 Zweig kann der bereitgestellte Inhalt des Stapelspeichers jeweils durch weitere Verarbeitungsstufen verarbeitet werden:



Durch Fortsetzen dieses Prinzips können tief verschachtelte Strukturen entstehen.

```
+ Input liest einen Wert von der Tastatur ein und legt ihn auf den Stack
λinput J gib eine ganze Zahl z ein. Der absolute Betrag |z| wird ermittelt ! Y

+ Falls keine Eingabe erfolgte (Abbruch), weiter im Sowilo Zweig
λ print K1 J Die Eingabe wurde abgebrochen Y Y

+ Eine Eingabe wurde erfolgreich durchgeführt: weiter im Siegel Zweig
λ λa_gt_b K0 + Vergleichsoperator 0 > eingabe
λ print J Fehler: Der Wert auf dem Stack ist keine Zahl und kann nicht
verglichen werden ! Y Y
λ + Auf dem Stack liegt das Ergebnis von 0 > eingabe
λ ifElse

+ Wenn eingabe < 0 ist, dann mit -1 multiplizieren
λ λmul K-1Y
```

```

    + Wenn eingabe >= 0 ist, dann mit 1 multiplizieren
    { λmul K1Y
      Y
    Y
  Y + Hier kann nun der absolute Betrag auf dem Stapel

  λprint K2 J Der absolute Betrag |z| = 1
  { print K1 J Der Stapel ist leer 1
  Y

```

Abrufen des Ergebnis- Stapelspeichers als Array $\mathbb{N}$

Der gesamte Stapelspeicher kann in einem λ , $\{$ und Υ Zweige als das spezielle Array $\mathbb{N}$ abgegriffen werden. Mittels $\mathbb{N}$ Parallel- Zugriffoperator können einzelne Elemente herausgegriffen und gezielt weiterbearbeitet werden. Zur Laufzeit wird jedes herausgegriffenen Element in einem eigenen *Laufzeittask* bearbeitet- der $\mathbb{N}$ ist damit das primäre Instrument zur Parallel- Programmierung. $\mathbb{N}$ hat folgende Signatur:

```

 $\mathbb{N}$  _Index1_ [_index2 [ ... [index n]]]
 $\lambda$  _meth_für_Zweig1_ + Methode, die auf den Wert mit Index 1 aus  $\mathbb{N}$  angewendet
wird
 $\lambda$  _meth_für_Zweig2_ + Methode, die auf den Wert mit Index 2 aus  $\mathbb{N}$  angewendet
wird
:
 $\lambda$  _meth_für_ZweigN_ + Methode, die auf den Wert mit Index N aus  $\mathbb{N}$  angewendet
wird
{ _meth_für_einen_outOfRange_Fehler_
Y _Folgefunktion_

```

Der Wert zu jedem Index wird an einen korrespondierenden λ Zweig geleitet, und kann dort mit einer Folge-Methode weiterbearbeitet werden.

Sollte ein Index außerhalb des Stapelspeicher- Array $\mathbb{N}$ liegen, dann wird kein λ Zweig betreten, sondern nur der $\{$ Zweig. In diesem kann eine Fehlerbehandlung stattfinden.

Υ wird in jedem Fall am Ende durchlaufen. Hier kann eine Folgefunktion gestartet werden. Im Kontext der parallelen *Laufzeittasks* stellt hier Υ einen *Join* dar.

Benennen von Ergebnissen einer Stufe

Alternativ zum Abruf und Weiterverarbeitung der Ergebnisse mit $\mathbb{N}\uparrow$ können die Einträge am Stufenausgang auch aus dem Stapelspeicher gelesen und benannt werden mit $\&$:

```

      +-----+
 $\lambda$  m a b c  a | b | c | -----+-----+ + Ablage der Parameter auf dem
Stapelspeicher
      +-----+-----+
      +-----+-----+

```

```

+--- h s      m(a, b, c) | <-+      |      + Ergebnis der Methode im h Zweig
bereitstellen
|
|      -----+
|      -----+
| +-- ξ e      m(a, b, c) | <-----+      + Ergebnis der Methode im ξ Zweig
bereitstellen
| |
| |      -----+
| |
| |      -----+
+--> Y ξres s(m(a, b, c)) oder e(m(a, b, c)) | + Ergebnis aus h oder ξ an
Namen res binden
      -----+

```

Das benannte Ergebnis kann dann im Folgenden weiterverwendet werden:

```

ξa K2
ξb K3

λsqu *a
Y ξaa

λsqu *b
Y ξbb

λadd *aa *bb
+ Weiterleiten des Ergebnisses an die Print- Methode. Achtung: Im AusgabeString
findet
+ String- Interpolation statt.
Y print J *a2 + *b2 = 1

```

Benennungen innerhalb von h und ξ Zweig

Eine Benennung inner halb eines h und ξ Zweiges ist nur lokal innerhalb dieses sichtbar.

```

ξaa K2

λsquRoot *aa
+ ξa_lok ist nur innerhalb des Siegel - Zweiges sichtbar
h ξa_lok λprint J *a_lok ist die Wurzel aus *aa 1
ξ λprint J *aa ist keine reele Quadratzahl! 1 Y
  λpush K0 Y
+ ξa_glob ist für den gesamten Kontext sichtbar, innerhalb dessen squRoot
aufgerufen wurde
Y ξa_glob

```

Benennen von Datenflussgraphen: Module M ... M

Komplett ausprogrammierte Datenflussgraphen können zwecks Wiederverwendung in Modul- Deklarationen eingeschlossen werden.

Eine Moduldeklaration ist ein Block, der zwischen $\mathbb{M}$ und $\mathbb{M}$ eingeschlossen wird. Dem $\mathbb{M}$ folgt der Name des Moduls:

```
 $\mathbb{M}$  moduleName
+ Hier wird der Wiederverwendende Datenflussgraph definiert.
 $\mathbb{M}$ 
```

Das Modul kann dann später wie eine elementare Datenverarbeitungsstufe mit den Zweigen $\mathbb{H}$ und $\mathbb{X}$ verwendet werden. Wann $\mathbb{H}$ und wann $\mathbb{X}$ aufgerufen werden, kann innerhalb des Moduls mit $\mathbb{H}\uparrow$ und $\mathbb{X}\uparrow$ definiert werden:

```
 $\mathbb{M}$  divKardinal
+ Hier wird der Wiederverwendende Datenflussgraph definiert.
 $\mathbb{A}$ pop  $\diamond$   $\mathbb{K}\uparrow$   $\mathbb{K}\uparrow$ 
 $\mathbb{X}$   $\mathbb{A}$ push  $\mathbb{J}$  Err divKardinal  $\mathbb{Y}$ 
 $\mathbb{X}\uparrow$ 
 $\mathbb{Y}$   $\mathbb{X}$ Nom  $\mathbb{X}$ Denom

 $\mathbb{A}$ ifElse  $\diamond$   $\mathbb{K}\uparrow$ 

 $\mathbb{Y}$ 

 $\mathbb{M}$ 
```

Von der Laufzeitumgebung bereitgestellte Stufen

Die Laufzeitumgebung hat bereits eine Reihe von Stufen vordefiniert und implementiert.

Programende

Diese Stufe beendet in jedem Fall das Programm.

```
 $\mathbb{A}$ fin $\mathbb{Y}$ 
```

Fehlerlog

Diese Stufe gibt die als Eingang E1 vorliegende Meldung und den aktuellen Stapelspeicherinhalt in einem Fehlerlog aus. Nach Ausführung dieser Stufe ist der Stapelspeicher in genau dem gleichen Zustand wie vor der Stufe.

```
 $\mathbb{A}$ logErr  $\mathbb{J}$  FEHLERMELDUNG  $\mathbb{Y}$ 
```

Infolog

Diese Stufe gibt die als Eingang E1 vorliegende Meldung und den aktuellen Stapelspeicherinhalt in einem Info- Log aus. Nach Ausführung dieser Stufe ist der Stapelspeicher in genau dem gleichen Zustand wie vor der Stufe.

```
λlogInf J FEHLERMELDUNG Y
```

Stapelspeicher mit Werten füllen

Mit dieser Stufe kann eine Liste von Werten auf den Stapelspeicher gelegt werden. Eine weitere Bearbeitung der Werte auf dem Stapelspeicher findet nicht statt.

Nachfolgende Stufen können die unveränderten Werte auf dem Stapelspeicher dann weiterverarbeiten.

```

                Inhalt Stapelspeicher
                --+---+---+
λ push  a ... z  a |...| z|
                --+---+---+
                ←...  4   4
                ←...  ξ   4
                ↳  Y

```

Alternative Verarbeitung ifElse

Diese Stufe nutzt die Stuktur der alternativen Ausführungspfade 4 und ξ aus, um eine elementare Verzweigung in Abhängigkeit eines boolschen Wertes zu implementieren.

Der Eingangsparameter von **ifElse** muss ein boolscher Wert sein. Ist er True, dann wird der 4 Zweig, sonst der ξ ausgeführt. Am Ende wird wieder im Y Zweig zusammengeführt.

```

λifElse _boolscherEingang_
  4 _Folgestufe_if_TRUE_
  ξ _Folgestufe_if_FALSE_
  Y _FolgeStufe_von_ifElse_

```

Datenstrom- orientierte Ausgabe

Es können Ausgaben in Dateien erfolgen. Dazu sind diese in einer Stufe zuerst als Datenströme zu öffnen, und dann können Teile des Stapelspeichers in diese ausgegeben werden.

```

λout _name_output_Stream_ _E1_ ... _En_
ξ _Verarbeitungsstufe_falls_Ausgabe_scheitert_
Y

```

Interaktives Parsen von LLP

Es ist ein Editor für LLP zu implementieren, der den Benutzer aktiv bei der Eingabe unterstützt.

Nach jedem vollständig eingegeben Wort kann z.B. der Parser gestartet werden.

Z.B. folgende Sitzung:

⌘ _

Der Parser erkennt das Prefix für semantische Beziehungen. Nun kann die Definition oder die Abfrage einer semantischen Beziehung folgen.

⌘ ↑ _

[#1] ↑ - semantische Beziehung abfragen [#2] ⌘ - semantische Beziehung definieren: _NID_Referring

Nachdem [#1] gewählt wurde, ist nun eine der möglichen semantischen Beziehungen auszuwählen

⌘ ↑ ⌘ isInstanceOf

[#1] ⌘ isPartOfSemContext [#2] ⌘ isInstanceOf [#3] ⌘ isPartOf [#4] ⌘ isSubTermOf [#5] ⌘ isSubNamespace

Nachdem [#2] gewählt wurde, gibt es eine große Auswahl von Namenscontainern, die Klassennamen von Klassen darstellen, mit denen andere Namenscontainer in der Beziehung ****isInstanceOf**** stehen können. Hier gibt es verschiedene Strategien, um den gesuchten Namenscontainer des Klassennamens zu finden:

1. Über den Namensraum- Pfad zur Naming ID des gesuchten Namenscontainers navigieren.

1. Es werden nur Namensraum- Pfade unterstützt, die Klassennamen enthalten

2. Es werden alle Namensraumpfade unterstützt. In einem Namensraum werden nur Namenscontainer von Klassennamen angezeigt.

2. Über ein Autocomplete- Textcontrol, das nur die CNT's von Klassennamen unterstützt, den CNT auswählen lassen.

Sei Variante 2 der Standard- Modus. In Variante 1 kann bei Bedarf umgeschaltet werden:

⌘ ↑ ⌘ isInstanceOf ⌘ milProgram

[#1] Select Naming- Containear with class name via Namespace